

Современные графические ускорители выполняют большое количество мелких задач одновременно. В отличие от центрального процессора (CPU), который предназначен для последовательного выполнения комплексных задач, графические ускорители спроектированы так, чтобы те выполняли множество небольших и независимых вычислений, таких к примеру, как отрисовки пикселей или обучение нейросетей. Одну такую мелкую задачу может выполнять специализированный блок, который называют ядром графического ускорителя. В современных видеокартах (GPU), количество ядер может скалироваться от нескольких тысяч до нескольких десятков тысяч.

Различие количества ядер у CPU и GPU является в том, что у центральный процессор выполняет большой спектр различных, сложных и разнообразных задач, последовательно, в то время, как ядро GPU является узкоспециализированным ядром, который может выполнять лишь определенный список задач, которые приходят на обработку. Также, и объем задач сильно отличается, из-за чего ядро CPU обладает большим размером кеша, чем ядро GPU. Чем меньше логики у аппаратного блока и чем меньше для него создано памяти для хранения временных и локальных данных и инструкций, тем меньше размер аппаратного блока. Следовательно размер одного ядра CPU намного меньше ядра GPU.

В отличие от ядер CPU, ядра GPU выполняют параллельно похожие операции над данными. В компьютерной графике, мультимедиа, криптографии и машинном обучении часто происходит ситуация, когда нужно работать с множествами данных, используя одну инструкцию. Взять в пример вычисление какой-либо картинки на экране. Сегодня разрешения экранов достигают более 2 миллионов пикселей, что представляет собой уже огромный пласт работы, который необходимо обрабатывать меньше чем за секунду. Поэтому, в графических ускорители обладают маленькими, но многочисленными ядрами, а также в них используется такая технология, как SIMD (Single Instruction, Multiple Data), что означает в переводе “одна инструкция, множество данных”. Использование принципа SIMD позволяет обеспечить параллелизм на уровне данных, позволяя работать с большим количеством информации, над которой требуется работа лишь одной конкретной инструкции [1]. Технология SIMD не только улучшает производительность вычислительных систем, но и уменьшает площадь, так как количество необходимых шин, для того чтобы связать аппаратные блоки существенно уменьшается.

SIMD не используется на всех возможных ядрах графического процессора. Он используется на кластере ядер, притом, что все ядра графического ускорителя могут отличаться друг от друга, в зависимости от выполняемых задач. На рис 1. можно увидеть архитектуру графического процессора, у которого существуют различные типы ядер:

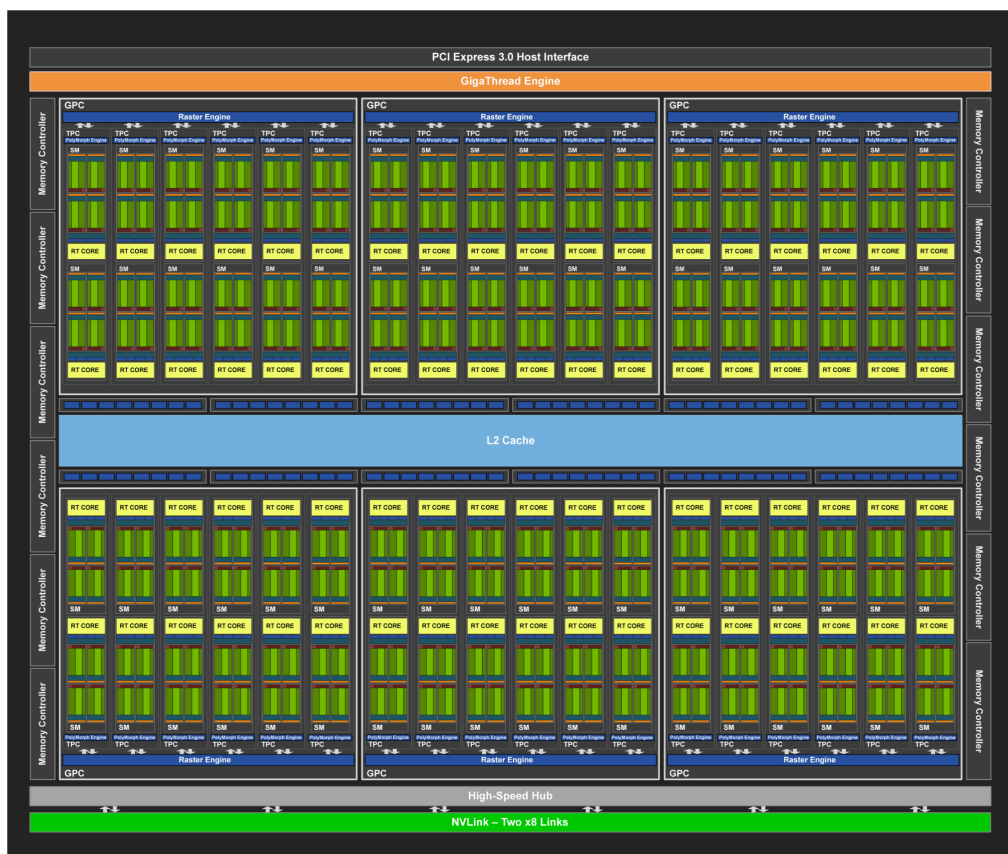


Figure 1: Одна из архитектур графического процессора

Лишь на одной архитектуре графического процессора можно увидеть около 4608 универсальных ядер, 72 ядер для выполнения рей-трейсинга и 576 тензорных ядер [2]. Один графический процессор может иметь 6 вычислительных графических кластеров, в каждом из которых по 12 потоковых мультимикропроцессоров. В каждом микропроцессоре есть 1 ядро для выполнения рей-трейсинга и 4 вычислительных единицы, в каждом из которых есть по 16 вычислительных универсальных ядер и 2 тензорных ядра.

Для разработки графического ускорителя, необходимо в первую очередь разработать вычислительное ядро, которое сможет обрабатывать входящие данные на параллельном уровне. Каждое ядро может иметь в свою очередь повторяющиеся компоненты вычислительного ядра, называемые нитями. Все нити в одном ядре выполняют одинаковые инструкции, с различными данными, параллельно работая уже на уровне ядра.

Перед разработкой нити нужно определиться, что необходимо в ней, для необходимой работы. В первую очередь, для разработки 1 нити, нужно обладать некоторым вычислительным аппаратным блоком, который может работать с числами. В компьютерной графике в основном используются для вычислений числа с плавающими точками. Поэтому, арифметически-логическое устройство (ALU) не подойдет, как основной вычислительный компонент для нити. Для работы с вещественными числами необходимо другой компонент, который называется математический сопроцессор (FPU). С помощью него, можно производить различные операции над числами с плавающей точкой. Такие операции могут быть не только, как сложение или умножение, но и взятие под корень, логарифмирование, сравнение и другие прочие операции, необходимые для работы с такими типами чисел. В аппаратном решении, FPU это объемный блок, который вычисляет числа аппаратным образом конвейерно. Благодаря конвейеризации можно достичь повышения производительности за счёт возможности работы с несколькими инструкциями на разных этапах одновременно [3].

При этом, хранение целочисленных чисел и вещественных существенно различается. Если для целочисленных чисел используется хранение в двоичном виде в представлении возведении степени двойки, то вещественные числа хранятся кардинально другим образом. Хранение и работа с вещественными числами строго описана в спецификации IEEE Std 754™ или же IEEE Standard for Floating-Point Arithmetic (Стандарт для арифметики с числами с плавающей точкой). Пример хранения вещественного числа можно увидеть на Рис. 2:

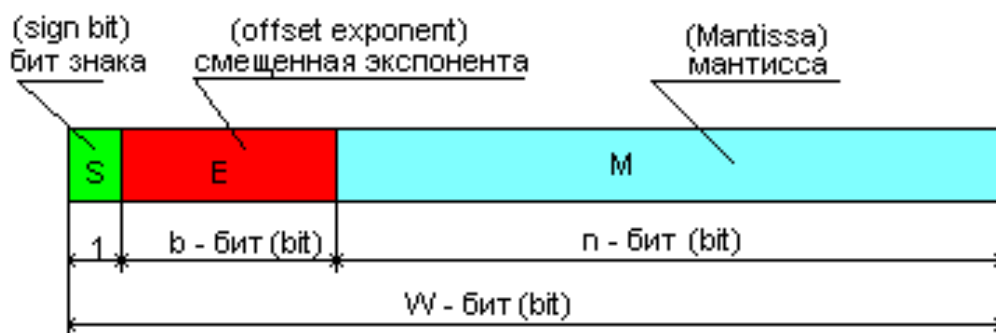


Figure 2: Одно из возможных представлений хранения вещественного числа

В данном стандарте числа биты, выделенные под вещественное число группируются на различные области, с помощью которых можно хранить число. Этими областями называются: бит знака, смещенная экспонента и мантисса. Поэтому, целочисленные ячейки (int) для хранения чисел

отличаются от хранения вещественных (float), а перевод чисел int во float и является нетривиальной задачей.

При работе с вещественными числами, конечно же возникает вопрос их хранения. Если хранение чисел в целочисленном формате, что в 32 битах памяти, что в 64 битах не особо различающимся, то с вещественными числами хранение числа в 32 битах и 64 может отличаться, так как выделение битов под мантиссу и экспоненту не является пропорциональным, в зависимости от бита.

Чем меньше размер вещественного числа, тем меньше возможная его точность, но при этом хранение и работа с таким вещественным числом упрощается. На уровне микроконтроллеров и решений, где не нужны точные решения, а нужна компактность и быстрота, там вещественные числа выгоднее хранить в 32-битном формате данных с плавающей точкой.

Сама FPU не хранит вещественные числа, поэтому, необходимо разработать в аппаратном блоке компонент, где FPU могла бы брать числа для вычислений и выкладывать результат. Таким компонентом является Регистровый файл. Это такой модуль, в котором хранятся регистры, которые могут хранить в себе данные для работы нити. Современные регистровые файлы представляют собой многопортовый массив быстрой статической памяти SRAM [4]. Регистровый файл обеспечивает быстрое обращение к данным, что особенно важно для графического ускорителя.

В начале работы графического ускорителя, FPU нужно подать данные и инструкцию. Данные (или же числа) можно положить в регистровый файл. Аппаратный блок, который забирает и выкладывает данные в регистровый файл называется LSU (Load-store unit) или же блоком работы с памятью. Это специализированный исполнительный блок, который отвечает за выполнение всех инструкций загрузки и сохранения [5]. С помощью него можно заполнить регистровый файл нити необходимыми числами для вычисления и выгрузить результат работы нити из регистрового файла в общую память. LSU также может не только просто брать значения и выкладывать их по адресу, но и также генерировать адреса, взяв значения для вычислений из регистрового файла или общей памяти.

Все вышеперечисленные компоненты, кроме LSU являются совокупностью нити. Таких нитей может быть несколько в ядре, но каждое из них выполняет одну и ту же инструкцию с различными входными

данными. Инструкция В памяти представляет собой набор битов, которые надо дешифровать перед работой с нитью. Для этого существует аппаратный блок, который называется декодером инструкций. Декодер инструкций преобразует машинный код, разделяет его на отдельные логические части и преобразует в управляющие сигналы. У всех нитей декодер общий, что образует модель SIMT (Single instruction, multiple threads или же “одна инструкция, много потоков”). Декодер инструкций передаёт команду нескольким нитям, чтобы все они при необходимости выполняли одновременное и полностью независимое параллельное выполнение одной и той же инструкции. При этом, каждая нить имеет свои независимые регистры данных и адреса, а также свою независимую память [6]. После декодировки инструкции нить уже передаёт работу либо LSU, либо FPU на выполнение. Для декодировщика инструкций также нужен блок для вызова инструкций.

Собрав все аппаратные блоки воедино, можно получить архитектуру, показанную на рис. 3:

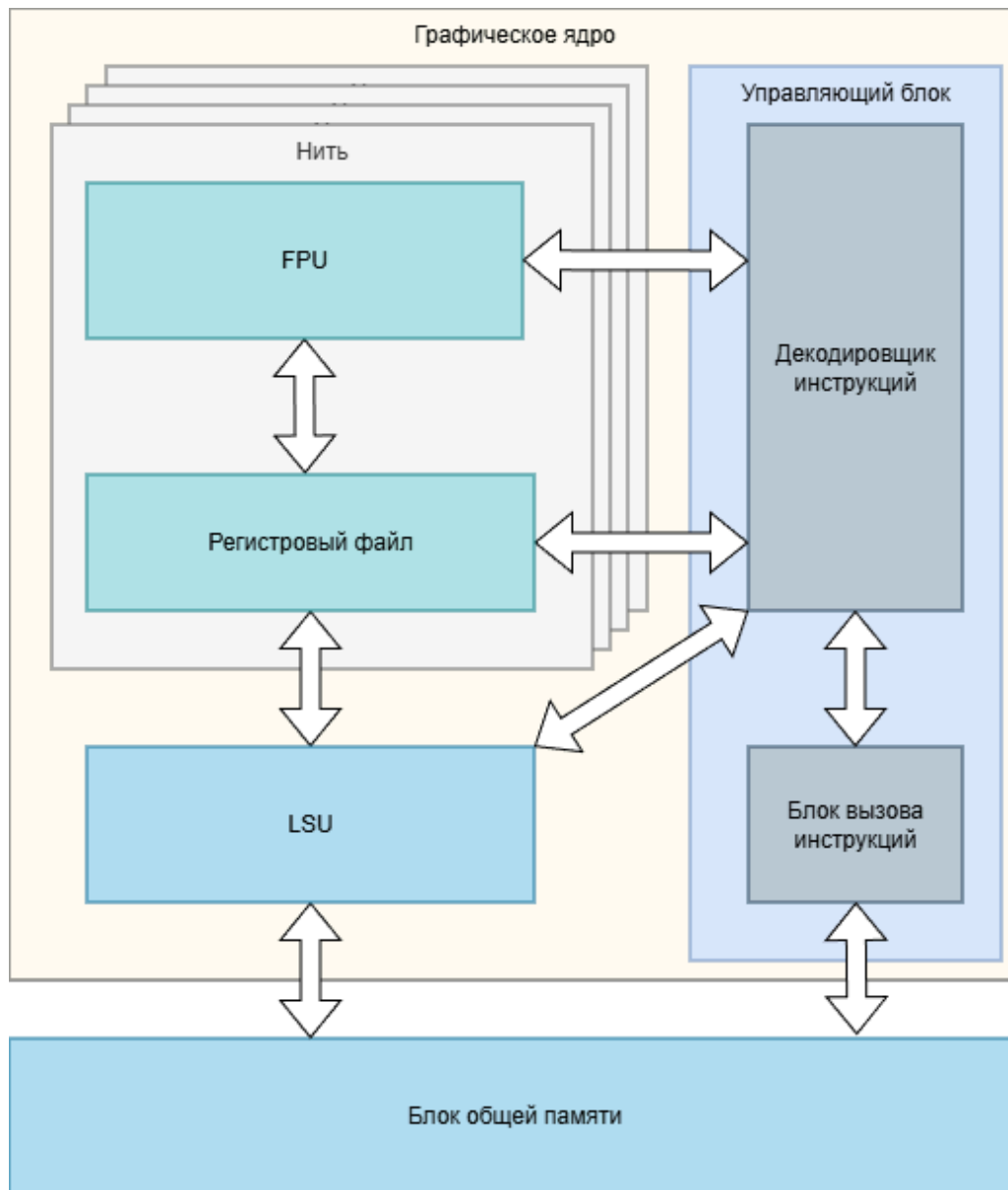


Figure 3: Архитектура графического ядра

Именно такая архитектура будет являться начальной для разрабатываемого графического ускорителя. Для одного ядра будет использоваться 4 нити, в каждом из которых будет по FPU и регистровому файлу.

Для разработки графического ядра, нужно учесть не только список компонентов, необходимых для корректной работы ядра, но также нужно определить шины взаимодействия между блоками.

Для FPU достаточно знать операцию, проводимую с числами, адреса в регистровом файле, где находятся числа над которыми будет производиться операция, а также адрес в регистровом файле, куда необходимо положить получившееся значение. Тип операции с декодировщика инструкций можно напрямую провести к FPU, а адреса вывести так, чтобы выходные порты регистрового файла выводили значения из указанных регистров по

адресу. При получении результата, адрес результата также направляется к регистровому файлу, чтобы регистровый файл записывал значения результата в регистр, с указанным адресом.

Регистровый файл помимо вывода портов к FPU, должен также принимать данные и выдавать их для LSU. Для LSU будет достаточно простой шины, где будет указан направление операции (чтение это или запись), адрес к какому регистру обратится и два порта выхода с датой: один на выход, другой на вход. Помимо этого стоит добавить сигнал готовности (ready) и сигнал валидности (valid). Логика сигнала готовности результата нити, придется связать с логикой готовности результата FPU: FPU сам по себе говорит о том, что данные готовы и он их выводит из своих портов на выход. Но для того, чтобы загрузить результат в регистровый файл, нужно какое-то время на защелкивание данных. Поэтому напрямую сигнал готовности FPU не соответствует сигналу готовности нити. Для того, чтобы правильно соединить логику рукопожатия FPU и LSU, нужно разработать отдельный подблок, который будет представлять из себя машину состояний (FSM), которая сообщает состояние FPU на данный момент и формирует сигналы в нити в зависимости от состояния FPU.

LSU должен иметь доступ не только к регистровому файлу, но и к блоку общей памяти. Шина связи между LSU и регистровой памятью не отличается от шины LSU и блоком общей памяти. LSU сама по себе ничего не будет делать, поэтому для её работы ей нужно также сформировывать инструкции. Инструкции также, как и к FPU приходят от декодировщика. В простом случае для LSU нужно было бы знать, откуда взять данные и куда положить. Но в аппаратном мире, LSU может также не только знать про данные, но и про адреса. Допустим, LSU может потребоваться не только брать-положить данные, но и брать адрес, формировать с помощью них новые адрес и уже по-новому адресу брать значение из общей памяти или регистрового файла. При этом, иногда не очень выгодно вообще обращаться к памяти по какому-то конкретному адресу, особенно при обращении к общей памяти: LSU придётся ждать, пока общая память сформирует данные по адресу и выставит сигнал, что данные готовы. Поэтому в архитектурах RISC-V инструкции зачастую не только хранят операции что и как сделать, но и еще иногда хранят immediate-поля, в которых будут находиться и данные [7]. Это удобно, потому что все инструкции также хранятся в общей памяти, что означает, что размер инструкции равен, размеру регистра. Из-за этого появляется место под

константные значения в виде `immediate`-полей, с помощью которых можно уже работать с ядром, не обращаясь к блоку общей памяти лишний раз. Пример инструкций архитектуры RISC-V можно увидеть на рис. 4:

32-bit RISC-V Instruction Formats																																		
Instruction Formats	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Register/register	funct7							rs2					rs1					funct3			rd				opcode									
Immediate	imm[11:0]												rs1					funct3			rd				opcode									
Upper Immediate	imm[31:12]																				rd				opcode									
Store	imm[11:5]							rs2					rs1					funct3			imm[4:0]				opcode									
Branch	[12]	imm[10:5]							rs2					rs1					funct3			imm[4:1]				[11]	opcode							
Jump	[20]	imm[10:1]											[11]	imm[19:12]							rd				opcode									
• opcode (7 bit): partially specifies which of the 6 types of <i>instruction formats</i> • funct7 + funct3 (10 bit): combined with opcode, these two fields describe what operation to perform • rs1 (5 bit): specifies register containing first operand • rs2 (5 bit): specifies second register operand • rd (5 bit): Destination register specifies register which will receive result of computation																																		

Figure 4: Примеры инструкций архитектуры RISC-V

Если реализовывать логику с `immediate`-полями, то в декодировщике инструкций гарантированно будет приходить набор битов, равный размеру регистра. По сути, декодировщик будет получать от блока вызова инструкций набор битов, дешифровать его в инструкцию и направлять инструкцию в нить или в LSU. Сам блок вызова инструкций будет брать эти инструкции в формате машинного кода из блока общей памяти.